

CS367 Software Analysis — Bytecode Worksheet

Name: _____

Exercise 1 — Tracing the JVM stack

As the JVM runs, it keeps an **operand stack** (a working scratchpad) and a **local variable table** (named slots). We will trace two methods: a short one to warm up on *branches*, then a loop.

Warmup — a forward branch: `max(a, b)`

Slots: slot 0 = `this`, slot 1 = `a`, slot 2 = `b`.

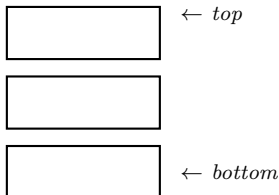
```
int max(int a, int b) {           0:  iload_1      5:  iload_1
    if (a > b)                     1:  iload_2      6:  ireturn
        return a;                 2:  if_icmple 7  7:  iload_2
    return b;                     (jumps)      8:  ireturn
}
```

We are calling `max(3, 8)`, so `a = 3`, `b = 8`.

Q1. `if_icmple 7` means “if the second-from-top $\leq$ top, jump to PC 7.” How many values does it pop off the stack?

Snapshot W1 — after PC 1: `iload_2`

Operand stack



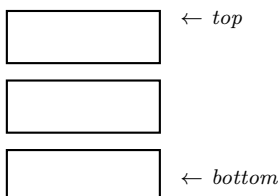
Local variables

slot 0: <code>this</code> =
slot 1: <code>a</code> =
slot 2: <code>b</code> =

`iload_1` then `iload_2` just ran. What is on the stack (top first)?

Snapshot W2 — after PC 2: `if_icmple 7`

Operand stack

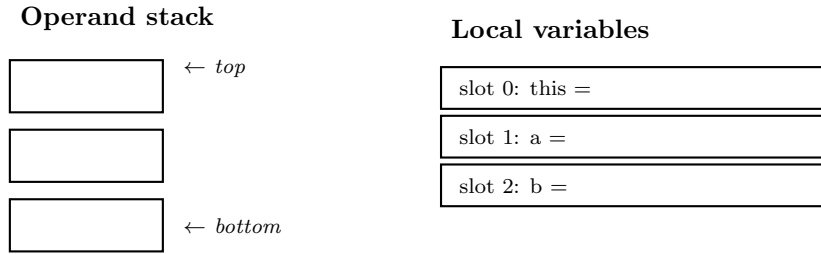


Local variables

slot 0: <code>this</code> =
slot 1: <code>a</code> =
slot 2: <code>b</code> =

The branch just executed. Is $3 \leq 8$ true? Did we jump or fall through, and what is on the stack now?

Snapshot W3 — after PC 7: `iload_2`



Show the stack just before `ireturn`. What value does `max(3, 8)` return?

Q2. The source says `if (a > b)`, but the bytecode tests `if_icmple ( $\leq$ )`. Why does the compiler branch on the *opposite* of the source condition?

The main event — a loop: `sum(n)`

Now the `sum` method and its bytecode. Same idea, but this method *loops*: watch for a jump that goes *backward*. The slots for this method are: slot 0 = `this`, slot 1 = `n`, slot 2 = `total`, slot 3 = `i`.

<code>public int sum(int n) {</code>	<code>0: iconst_0</code>	<code>9: iload_2</code>
<code> int total = 0;</code>	<code>1: istore_2</code>	<code>10: iload_3</code>
<code> for (int i = 1;</code>	<code>2: iconst_1</code>	<code>11: iadd</code>
<code> i <= n; i++) {</code>	<code>3: istore_3</code>	<code>12: istore_2</code>
<code> total += i;</code>	<code>4: iload_3</code>	<code>13: iinc 3, 1</code>
<code> }</code>	<code>5: iload_1</code>	<code>16: goto 4</code>
<code> return total;</code>	<code>6: if_icmpgt 19</code>	<code>19: iload_2</code>
<code>}</code>	<code>(jumps)</code>	<code>20: ireturn</code>

We are calling `sum(5)`, so `n = 5`.

Part A — Predict before you trace

Answer in one line each.

1. What two values does `if_icmpgt 19` compare, and where does it jump when the comparison is true?

2. Does `iinc 3, 1` push anything onto the operand stack? What does it change?

3. After `goto 4` runs the first time, what are the values of `total` and `i`?

Part B — Fill in the snapshots

At each moment below, draw the operand stack (write each value in a cell, leave unused cells empty) and fill in the local variable slots. The instruction that *just executed* is named at the top of each frame.

Snapshot 1 — after PC 3: `istore_3` (loop about to start)

Operand stack		Local variables
	← <i>top</i>	slot 0: this =
		slot 1: n =
		slot 2: total =
	← <i>bottom</i>	slot 3: i =

Both loop variables have just been initialized. What is on the stack right now?

Snapshot 2 — after PC 5: `iload_1` (first time through)

Operand stack		Local variables
	← <i>top</i>	slot 0: this =
		slot 1: n =
		slot 2: total =
	← <i>bottom</i>	slot 3: i =

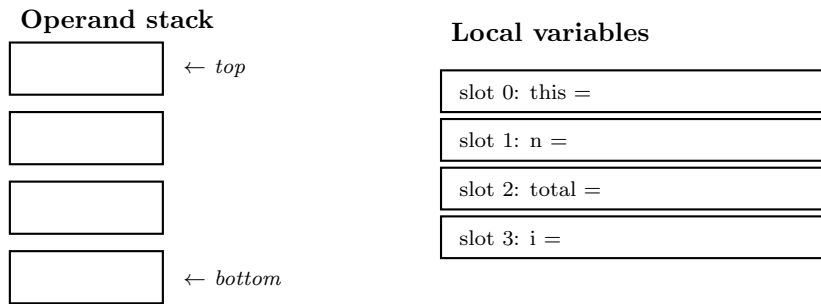
The comparison `if_icmpgt` is about to run. What did `iload_3` then `iload_1` leave on the stack?

Snapshot 3 — after PC 6: `if_icmpgt 19` (first time through)

Operand stack		Local variables
	← <i>top</i>	slot 0: this =
		slot 1: n =
		slot 2: total =
	← <i>bottom</i>	slot 3: i =

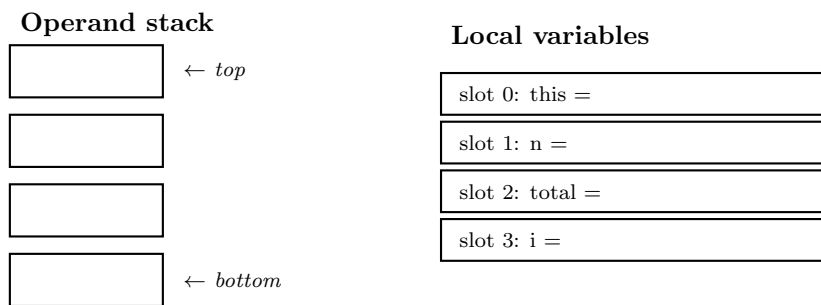
The branch just ran and did *not* jump (1 is not > 5). What did it pop — what is on the stack now?

Snapshot 4 — after PC 11: `iadd` (first iteration)



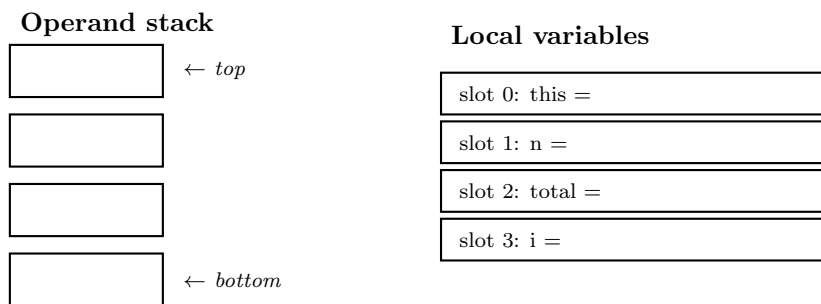
The body just computed `total + i`. What single value is on the stack, and what is still in the slots?

Snapshot 5 — after PC 13: `iinc 3, 1` (first iteration)



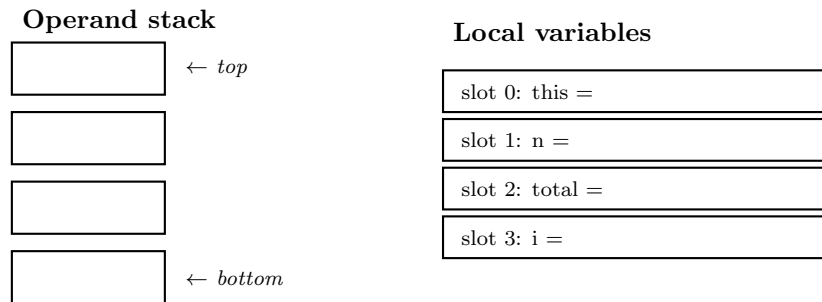
The counter was just updated. Did the stack change? What is `i` now?

Snapshot 6 — after PC 5: `iload_1` (SECOND time through)



We have looped back via `goto 4`. Show the stack and slots at the top of the second iteration.

Snapshot 7 — after PC 19: `iload_2` (loop finished)



The loop test finally failed and we jumped to 19. What is about to be returned?

Part C — Zoom out

4. How many times does the loop body (PC 4–16) run for `sum(5)`? What is the final return value?

5. Which instruction is the loop-counter update, and which is the loop-exit test? Why is the test at the *top* of the loop with the jump-back at the bottom?

Exercise 2 — From bytecode back to source

Each snippet below is the bytecode for a small method. Write Java source that could have produced it. There may be more than one right answer — the structure just has to match. Slot assignments are given where they help.

Problem 1

Slots: 0 = `this`, 1 = `a`, 2 = `b`. Method returns `int`.

```
0: iload_1
1: iload_2
2: isub
3: ireturn
```

Your source:

Problem 2

Slots: 0 = `this`, 1 = `x`. Method returns `boolean`.

```
0: iload_1
1: ifle 6
4: iconst_1
5: ireturn
6: iconst_0
7: ireturn
```

Hint: `ifle 6` means “if the value is ≤ 0 , jump to 6.” `iconst_1` pushes `true`, `iconst_0` pushes `false`. What boolean expression is true only when `x > 0`?

Your source:

Problem 3

Slots: 0 = `this`, 1 = `n`, 2 = `result`. Method returns `int`.

```
0: iconst_1
1: istore_2
2: iload_2
3: iload_1
4: imul
5: istore_2
6: iload_2
7: ireturn
```

Your source:

Problem 4 — challenge

Slots: 0 = `this`, 1 = `x`. Method returns `int`.

```
0: iload_1
1: ifge 7
4: iload_1
5: ineg
6: istore_1
7: iload_1
8: ireturn
```

Hints: `ineg` pops one value and pushes its negation. `ifge 7` means “if the value is ≥ 0 , jump to 7.” Notice the branch *skips over* PCs 4–6 and both paths rejoin at PC 7. What does this method compute?

Your source:
